

Cửa an ninh xác thực bằng khuôn mặt

YOLOv11-Face · ArcFace · Liveness Detection · ESP32-CAM

Nhóm thực hiện

Lưu Huy Minh Quang	23127016
Hoàng Nhân	23127094
Trần Cao Vân	23127141
Nguyễn Trần Thiên Phú	23127246

Khoa CNTT – ĐH Khoa học Tự nhiên, ĐHQG TP.HCM
Môn: Nhập môn lập trình điều khiển thiết bị thông minh

Tháng 5, 2026

Nội dung trình bày

- 1 Giới thiệu đề tài và bài toán
- 2 Lựa chọn công nghệ và lý do
- 3 Phương pháp đề xuất
- 4 Kiến trúc hệ thống

Phần 1

Giới thiệu đề tài và bài toán

Sản phẩm đề xuất

Cửa an ninh thông minh – xác thực danh tính bằng khuôn mặt, tự động mở/ khóa cửa thông qua hệ thống ESP32-CAM + Server AI.

Ý tưởng:

- ESP32-CAM gắn tại cửa, chụp ảnh khuôn mặt
- Server nhận dạng bằng AI (YOLOv11 + ArcFace)
- Kiểm tra mặt thật bằng liveness detection (nháy mắt)
- Khớp danh tính → mở cửa; không khớp → từ chối + cảnh báo

Tại sao dùng AI face embedding?

So sánh pixel trực tiếp nhạy với ánh sáng, góc chụp và biểu cảm. ArcFace mã hóa khuôn mặt thành vector 512 chiều – chỉ giữ lại danh tính, bỏ qua nhiễu.

Ứng dụng:

- Nhà thông minh (smart home)
- Văn phòng, phòng server, phòng lab
- Chung cư, ký túc xá
- Mở rộng: chấm công, điểm danh

Đầu vào (Input)

- Ảnh 2D từ ESP32-CAM (640×480)
- Có thể mờ, nhiễu, thiếu sáng
- Có thể góc nghiêng, nhiều người

Đầu ra (Output)

- Kết quả: PASS hoặc DENY
- Lệnh relay mở/khóa cửa
- Log nhật ký ra/vào

Ràng buộc kỹ thuật

- ESP32-CAM giới hạn RAM/CPU → AI phải chạy trên server
- Ảnh độ phân giải thấp → cần detect/crop đúng vùng mặt
- Dữ liệu sinh trắc cần bảo mật
- Phản hồi nhanh (< 3 giây)
- Chống giả mạo bằng ảnh in/video

Phần 2

Lựa chọn công nghệ và lý do

Tại sao chọn ArcFace?

Giải pháp	Ưu điểm	Nhược điểm	Chọn?
dlib (HOG)	Nhẹ, chạy CPU	Accuracy thấp, nhạy góc	Không
FaceNet	Embedding 128-d tốt	Train phức tạp	Không
DeepFace	Wrapper tiện	Nặng, khó customize	Không
ArcFace	512-d, 99.8% LFW	Cần GPU inference	Có

Tại sao phù hợp đề án?

- Pretrained trên 5.8 triệu ảnh (MS1MV2) → **không cần train lại**
- Embedding 512 chiều bất biến với ánh sáng, góc, biểu cảm
- Thư viện insightface dễ dùng, hỗ trợ Python tốt
- Chỉ cần inference trên laptop/server
- ESP32-CAM chỉ gửi ảnh, không cần xử lý nặng

Tại sao chọn YOLOv11-Face?

YOLOv11-Face – Detect khuôn mặt

Vai trò: tìm và crop mặt trước khi đưa vào ArcFace

- Nhanh, real-time trên server
- Xử lý nhiều mặt trong 1 frame
- Crop chính xác giúp ArcFace hoạt động tốt hơn
- Tốt hơn MTCNN (cũ) và dlib (chậm hơn)

Kết hợp

YOLOv11 tìm mặt → crop → resize 112×112 → đưa vào ArcFace.

Tại sao cần Liveness Detection?

Liveness Detection – Chống giả mạo

Vai trò: đảm bảo mặt thật, không phải ảnh in hoặc video phát lại

- Dùng Eye Aspect Ratio (EAR) – phát hiện nháy mắt
- Không cần thêm phần cứng (camera IR, depth sensor)
- Đơn giản nhưng hiệu quả với ảnh in
- Có thể mở rộng: quay đầu, mở miệng

Tóm lại

Cả 3 thành phần (YOLOv11 + ArcFace + Liveness) đều có pretrained/code sẵn, không cần train from scratch, phù hợp đồ án với ESP32-CAM.

Phần 3

Phương pháp đề xuất

4 bước xử lý

- ① ESP32-CAM chụp ảnh, gửi lên server
- ② YOLOv11 detect + crop khuôn mặt
- ③ ArcFace trích xuất embedding 512 chiều
- ④ Liveness check + cosine matching với database

Phương pháp – Đăng ký khuôn mặt

Quy trình đăng ký (1 lần cho mỗi user)

- 1 Chụp 5–10 ảnh user qua ESP32-CAM (nhiều góc, ánh sáng)
- 2 YOLOv11 detect + crop mặt từng ảnh
- 3 ArcFace trích xuất embedding 512-d cho mỗi ảnh
- 4 Tính embedding trung bình $\rightarrow$ lưu vào database

Quy trình xác thực (mỗi lần vào cửa)

- 1 ESP32-CAM chụp ảnh, gửi server
- 2 Detect + crop + trích embedding
- 3 Kiểm tra liveness (nháy mắt)
- 4 So cosine similarity với DB; nếu $\geq \theta \rightarrow$ PASS

Phương pháp – ArcFace hoạt động thế nào?

Cách hoạt động:

- Mã hóa khuôn mặt thành vector 512 chiều
- Cùng 1 người $\rightarrow$ vector gần nhau (cosine $\approx 0.7-0.9$)
- Khác người $\rightarrow$ vector xa nhau (cosine < 0.3)
- Bất biến với ánh sáng, góc, biểu cảm

Công thức xác thực:

$$\text{sim} = \frac{e_{\text{input}} \cdot e_{\text{DB}}}{\|e_{\text{input}}\| \|e_{\text{DB}}\|}$$

$\text{sim} \geq 0.5 \rightarrow \text{PASS}$ (mở cửa)

$\text{sim} < 0.5 \rightarrow \text{DENY}$ (từ chối)

Phương pháp – Tại sao không so pixel?

So pixel trực tiếp

- Pixel thay đổi khi ánh sáng đổi
- Pixel thay đổi khi góc chụp khác
- Pixel thay đổi khi biểu cảm khác
- → Không ổn định, dễ sai

Face embedding (ArcFace)

- Embedding **chỉ giữ lại danh tính**
- Bỏ qua ánh sáng, góc, biểu cảm
- Ảnh 112×112 sau crop là đủ
- → Ổn định, chính xác cao

Phù hợp ESP32-CAM

Ảnh 640×480 chất lượng thấp vẫn hoạt động tốt vì ArcFace chỉ cần vùng mặt 112×112 sau crop.

Vấn đề: Ai cầm ảnh in hoặc phát video khuôn mặt lên → ArcFace vẫn match.

Giải pháp: Eye Aspect Ratio (EAR)

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

- Mắt mở: $EAR \approx 0.25-0.30$
- Đang nháy: $EAR < 0.2$
- Ảnh in: EAR cố định, không bao giờ thay đổi → phát hiện giả

Flow kiểm tra liveness

- 1 LED/LCD hiển thị “Hãy nháy mắt”
- 2 Server theo dõi EAR trong 2–3 giây
- 3 Phát hiện nháy → mặt thật → tiếp tục xác thực
- 4 Không nháy trong 3s → nghi giả mạo → DENY

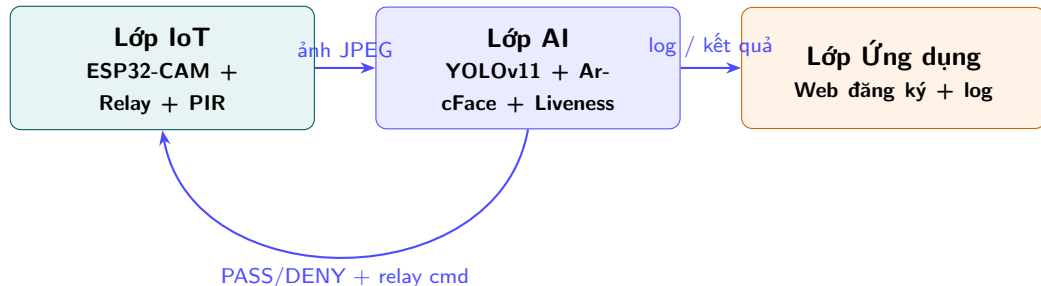
Mở rộng (nếu đủ thời gian)

- Random challenge: “quay trái”, “mở miệng”
- Phát hiện texture giấy/màn hình
- Kiểm tra chuyển động tự nhiên

Phần 4

Kiến trúc hệ thống

Kiến trúc tổng thể – 3 lớp



Phân chia trách nhiệm

ESP32-CAM chỉ chụp ảnh và điều khiển relay. Toàn bộ AI chạy trên laptop/server.

ESP32-CAM (IoT)

- Chụp ảnh
- Gửi HTTP POST
- Nhận lệnh relay
- LED/Buzzer báo

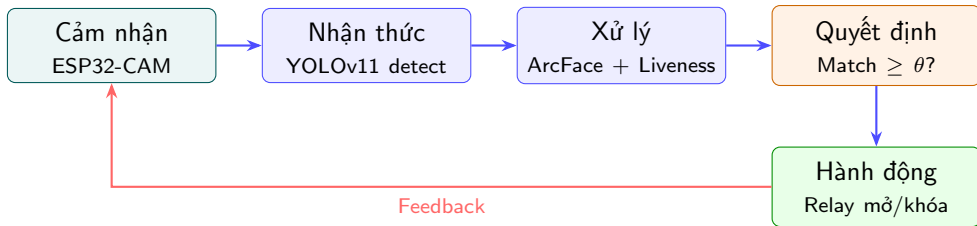
Laptop/Server (AI)

- YOLOv11 detect
- ArcFace embedding
- Liveness check
- Cosine matching

Web Dashboard

- Đăng ký khuôn mặt
- Quản lý user
- Xem nhật ký
- Cấu hình ngưỡng

Vòng lặp điều khiển thông minh



Đặc trưng “điều khiển thiết bị thông minh”

- Vòng lặp kín: cảm biến → AI xử lý → thiết bị thực thi → phản hồi
- Nếu ảnh kém chất lượng → yêu cầu chụp lại (feedback)
- Nếu liveness fail → yêu cầu nháy mắt lại (retry)
- Hệ thống tự ra quyết định dựa trên ngưỡng đã cấu hình

-  J. Deng *et al.*, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” *CVPR*, 2019.
-  Ultralytics, “YOLOv11,” GitHub, 2024.
-  T. Soukupova & J. Cech, “Eye Blink Detection Using Facial Landmarks,” *Computer Vision Winter Workshop*, 2016.
-  G. B. Huang *et al.*, “Labeled Faces in the Wild,” UMass Tech Report, 2007.
-  InsightFace, “InsightFace: 2D and 3D Face Analysis Project,” GitHub, 2024.

Cảm ơn!

Câu hỏi & Thảo luận